Part XV: API Reference

Version: 3.0
Date: January 30, 2026
Status: Complete (v3.0 Audit)

This document provides complete API documentation for the Luna Engine HTTP interface. All endpoints are served from the FastAPI server defined in `src/luna/api/server.py`.

Table of Contents

1. [Overview](#1-overview)
2. [Core Interaction Endpoints](#2-core-interaction-endpoints)
3. [Engine Status Endpoints](#3-engine-status-endpoints)
4. [Memory Operations Endpoints](#4-memory-operations-endpoints)
5. [Extraction Endpoints](#5-extraction-endpoints)
6. [History Management (Hub API) Endpoints](#6-history-management-hub-api-endpoints)
7. [Debug Endpoints](#7-debug-endpoints)
8. [Voice Integration Endpoints](#8-voice-integration-endpoints)
9. [Tuning Endpoints](#9-tuning-endpoints)
10. [Ring Buffer Endpoints](#10-ring-buffer-endpoints)
11. [System Endpoints](#11-system-endpoints)
12. [SSE Event Reference](#12-sse-event-reference)
13. [Error Handling](#13-error-handling)
14. [Examples](#14-examples)

---

1. Overview

Base URL

```
http://localhost:8000
```

The default port is `8000`. Configure via environment variable `LUNA_API_PORT`.

Authentication

No authentication is currently implemented. All endpoints are publicly accessible on localhost.

Content Types

Type	Usage
`application/json`	Request/response bodies
`text/event-stream`	SSE streaming responses

CORS Configuration

CORS is enabled for local development:

- `http://localhost:3000`
- `http://localhost:5173`
- `http://localhost:5174`
- `http://127.0.0.1:3000`
- `http://127.0.0.1:5173`
- `http://127.0.0.1:5174`

Streaming Endpoints

Four endpoints return Server-Sent Events (SSE):

Endpoint	Method	Description
`/stream`	POST	Token-by-token response streaming
`/persona/stream`	POST	Context-first streaming with memory
`/thoughts`	GET	Internal thought process monitoring
`/voice/stream`	GET	Voice system status updates

SSE responses include these headers:
```http
Content-Type: text/event-stream
Cache-Control: no-cache
Connection: keep-alive
X-Accel-Buffering: no
```

---

2. Core Interaction Endpoints

These endpoints handle direct communication with Luna.

POST /message

Send a message and receive a synchronous response.

Request Body:

Field	Type	Required	Default	Description
`message`	string	Yes	-	Message to send (1-10000 chars)
`timeout`	float	No	30.0	Response timeout in seconds (1.0-120.0)
`stream`	bool	No	false	Use streaming mode (ignored, use `/stream` instead)

Response:

Field	Type	Description
`text`	string	Luna's response text
`model`	string	Model used for generation
`input_tokens`	int	Input token count
`output_tokens`	int	Output token count
`latency_ms`	float	Response latency in milliseconds
`delegated`	bool	Whether request was delegated to Claude API

| `local` | bool | Whether local inference was used |
| `fallback` | bool | Whether fallback model was used |

curl Example:
```bash
curl -X POST http://localhost:8000/message \
  -H "Content-Type: application/json" \
  -d '{"message": "Hello Luna, how are you?"}'
```

Example Response:
```json
{
  "text": "Hello! I'm doing well, thank you for asking. How can I help you today?",
  "model": "qwen3b",
  "input_tokens": 45,
  "output_tokens": 18,
  "latency_ms": 1234.5,
  "delegated": false,
  "local": true,
  "fallback": false
}
```

POST /stream

Send a message and stream the response token-by-token via SSE.

Request Body:

Field	Type	Required	Default	Description
`message`	string	Yes	-	Message to send (1-10000 chars)
`timeout`	float	No	30.0	Response timeout in seconds (1.0-120.0)

SSE Events:

Event	Data Fields	Description
`token`	`text`	Generated token chunk
`done`	`model`, `input_tokens`, `output_tokens`, `latency_ms`, `delegated`, `local`, `fallback`	Generation complete
`error`	`error`	Error occurred

curl Example:
```bash
curl -X POST http://localhost:8000/stream \
  -H "Content-Type: application/json" \
  -H "Accept: text/event-stream" \
  -d '{"message": "Tell me a short joke"}'
```

Example Response Stream:
```
event: token
data: {"text": "Why"}

event: token
data: {"text": " did"}

event: token
data: {"text": " the"}

event: token
data: {"text": " scarecrow"}

event: done
data: {"model": "qwen3b", "input_tokens": 12, "output_tokens": 25, "latency_ms": 850.2,
"delegated": false, "local": true, "fallback": false}
```

POST /persona/stream

Stream Luna's response with context (memory and state) sent before tokens. This allows frontends to display relevant context while the response generates.

Request Body:

Field	Type	Required	Default	Description
`message`	string	Yes	-	Message to send (1-10000 chars)
`timeout`	float	No	30.0	Response timeout in seconds (1.0-120.0)

SSE Data Format (typed JSON, no named events):

Type	Fields	Description
`context`	`memory`, `state`	Memory items and engine state (sent first)
`token`	`text`	Generated token chunk
`done`	`response`, `metadata`	Full response text and generation metadata
`error`	`message`	Error message

curl Example:
```bash
curl -X POST http://localhost:8000/persona/stream \
  -H "Content-Type: application/json" \
  -H "Accept: text/event-stream" \
  -d '{"message": "What did we discuss yesterday?"}'
```

Example Response Stream:
```
```

```
data: {"type": "context", "memory": [{"id": "mem_001", "content": "Yesterday we discussed the project timeline...", "type": "CONVERSATION", "source": "session_abc"}], "state": {"session_id": "session_abc", "is_processing": true, "state": "RUNNING", "model": "qwen3b"}}

data: {"type": "token", "text": "Yesterday"}

data: {"type": "token", "text": " we"}

data: {"type": "token", "text": " discussed"}

data: {"type": "done", "response": "Yesterday we discussed the project timeline and agreed on the milestones.", "metadata": {"model": "qwen3b", "output_tokens": 45, "local": true}}
```

POST /abort

Abort the current generation.

Request Body: None

Response:

Field	Type	Description
`status`	string	`"aborted"` or `"no_generation"`
`message`	string	Status message

curl Example:
```bash
curl -X POST http://localhost:8000/abort
```

Example Response:
```json
{
  "status": "aborted",
  "message": "Generation aborted successfully"
}
```

POST /interrupt

Interrupt Luna's current processing. Triggers the agentic interrupt handler, which gracefully stops the current task.

Request Body: None

Response:

| Field | Type | Description |

|-------|------|-------------|
`status`	string	`"interrupted"` or `"no_task"`
`message`	string	Status message
`interrupted_goal`	string	Goal that was interrupted (if any)
`pending_messages`	int	Number of pending messages in queue

curl Example:
```bash
curl -X POST http://localhost:8000/interrupt
```

Example Response:
```json
{
  "status": "interrupted",
  "message": "Task interrupted",
  "interrupted_goal": "Research and summarize quantum computing",
  "pending_messages": 0
}
```

GET /thoughts

Stream Luna's internal thought process via SSE. Shows real-time progress during agentic processing.

Request Body: None

SSE Events:

Event	Data Fields	Description
`status`	`connected`, `is_processing`, `goal`	Initial connection status
`thought`	`type`, `message`, `is_processing`, `goal`	Internal thought/progress message
`phase`	`type`, `phase`, `is_processing`, `goal`	Processing phase change
`step`	`type`, `step_num`, `total_steps`, `description`, `is_processing`, `goal`	Plan step execution
`ping`	`is_processing`, `pending`	Keepalive (every 15 seconds)

curl Example:
```bash
curl -X GET http://localhost:8000/thoughts \
  -H "Accept: text/event-stream"
```

Example Response Stream:
```
event: status
data: {"connected": true, "is_processing": false, "goal": null}

event: thought
```

```
data: {"type": "thought", "message": "[DIRECT] Hello Luna...", "is_processing": true, "goal": "Respond to greeting"}

event: thought
data: {"type": "thought", "message": "[OK] local: 18 tokens", "is_processing": false, "goal": null}

event: ping
data: {"is_processing": false, "pending": 0}
```

---

3. Engine Status Endpoints

These endpoints provide visibility into Luna's current state and health.

GET /status

Get comprehensive engine status and metrics.

Query Parameters: None

Response:

Field	Type	Description
`state`	string	Engine state (`STARTING`, `RUNNING`, `STOPPING`, `STOPPED`)
`uptime_seconds`	float	Engine uptime in seconds
`cognitive_ticks`	int	Number of cognitive tick cycles
`events_processed`	int	Total events processed
`messages_generated`	int	Total messages generated
`actors`	list[string]	Active actor names
`buffer_size`	int	Current input buffer size
`current_turn`	int	Conversation turn counter
`context`	dict	Revolving context statistics
`agentic`	object	Agentic processing statistics

Agentic Stats Schema:

Field	Type	Description
`is_processing`	bool	Currently processing a task
`current_goal`	string	Current goal being worked on
`pending_messages`	int	Messages waiting in queue
`tasks_started`	int	Total tasks started
`tasks_completed`	int	Tasks completed successfully
`tasks_aborted`	int	Tasks aborted/interrupted
`direct_responses`	int	Direct (non-planned) responses
`planned_responses`	int	Responses requiring planning
`agent_loop_status`	string	Current agent loop status

curl Example:
```bash
```

curl http://localhost:8000/status
```

**Example Response:**
```json
{
  "state": "RUNNING",
  "uptime_seconds": 3600.5,
  "cognitive_ticks": 1250,
  "events_processed": 847,
  "messages_generated": 156,
  "actors": ["director", "matrix", "scribe", "librarian"],
  "buffer_size": 0,
  "current_turn": 42,
  "context": {
    "token_budget": 8000,
    "tokens_used": 2450,
    "items_count": 18
  },
  "agentic": {
    "is_processing": false,
    "current_goal": null,
    "pending_messages": 0,
    "tasks_started": 156,
    "tasks_completed": 154,
    "tasks_aborted": 2,
    "direct_responses": 142,
    "planned_responses": 14,
    "agent_loop_status": "idle"
  }
}
```

GET /health

Simple health check endpoint. Always returns 200 to indicate the API server is responsive.

Response:

Field	Type	Description
`status`	string	`"healthy"` or `"starting"`
`state`	string	Engine state name

curl Example:
```bash
curl http://localhost:8000/health
```

Example Response:
```json
{

```
  "status": "healthy",
  "state": "RUNNING"
}
```

GET /history

Get recent conversation history.

Query Parameters:

Parameter	Type	Default	Description
`limit`	int	20	Maximum messages to return

Response:

Field	Type	Description
`messages`	list	Conversation message objects
`total`	int	Total message count in history

Message Schema:

Field	Type	Description
`role`	string	`"user"` or `"assistant"`
`content`	string	Message content
`timestamp`	int	Unix timestamp

curl Example:
```bash
curl "http://localhost:8000/history?limit=5"
```

Example Response:
```json
{
  "messages": [
    {"role": "user", "content": "Hello Luna", "timestamp": 1737817200},
    {"role": "assistant", "content": "Hello! How can I help you today?", "timestamp":
1737817201},
    {"role": "user", "content": "What is the weather like?", "timestamp": 1737817230}
  ],
  "total": 42
}
```

GET /consciousness

Get Luna's current consciousness state including mood, coherence, and attention.

Response:

Field	Type	Description
`mood`	string	Current mood state
`coherence`	float	Coherence score (0.0-1.0)
`attention_topics`	int	Number of topics in attention
`focused_topics`	list[dict]	Currently focused topics with weights
`top_traits`	list[tuple]	Top personality traits with scores
`tick_count`	int	Consciousness tick counter
`last_updated`	string	ISO timestamp of last update

curl Example:
```bash
curl http://localhost:8000/consciousness
```

Example Response:
```json
{
  "mood": "curious",
  "coherence": 0.87,
  "attention_topics": 5,
  "focused_topics": [
    {"topic": "quantum computing", "weight": 0.85},
    {"topic": "user project", "weight": 0.72}
  ],
  "top_traits": [
    ["helpful", 0.95],
    ["curious", 0.88],
    ["thoughtful", 0.82]
  ],
  "tick_count": 1250,
  "last_updated": "2026-01-25T14:30:00Z"
}
```

---

4. Memory Operations Endpoints

These endpoints manage Luna's memory substrate directly.

POST /memory/nodes

Create a new memory node directly (bypasses extraction pipeline).

Request Body:

Field	Type	Required	Default	Description

`node_type`	string	Yes	-	Node type: `FACT`, `DECISION`, `PROBLEM`, `BELIEF`, `PREFERENCE`, `RELATIONSHIP`
`content`	string	Yes	-	Node content (min 1 char)
`source`	string	No	null	Source identifier
`confidence`	float	No	1.0	Confidence level (0.0-1.0)
`importance`	float	No	0.5	Importance level (0.0-1.0)

Response:

Field	Type	Description
`id`	string	Generated node ID
`node_type`	string	Node type
`content`	string	Node content
`source`	string	Source identifier
`confidence`	float	Confidence level
`importance`	float	Importance level
`access_count`	int	Access count (starts at 0)
`reinforcement_count`	int	Reinforcement count (starts at 0)
`lock_in`	float	Lock-in coefficient
`lock_in_state`	string	`drifting`, `fluid`, or `settled`
`created_at`	string	ISO timestamp

curl Example:
```bash
curl -X POST http://localhost:8000/memory/nodes \
  -H "Content-Type: application/json" \
  -d '{
    "node_type": "FACT",
    "content": "The user prefers dark mode interfaces",
    "confidence": 0.9,
    "importance": 0.7
  }'
```

Example Response:
```json
{
  "id": "node_abc123",
  "node_type": "FACT",
  "content": "The user prefers dark mode interfaces",
  "source": null,
  "confidence": 0.9,
  "importance": 0.7,
  "access_count": 0,
  "reinforcement_count": 0,
  "lock_in": 0.0,
  "lock_in_state": "drifting",
  "created_at": "2026-01-25T14:30:00Z"
}
```

GET /memory/nodes/{node_id}

Get a specific memory node by ID.

Path Parameters:

Parameter	Type	Description
`node_id`	string	Node identifier

Response: Same schema as POST /memory/nodes response

curl Example:
```bash
curl http://localhost:8000/memory/nodes/node_abc123
```

GET /memory/nodes

List memory nodes with optional filtering.

Query Parameters:

Parameter	Type	Default	Description
`node_type`	string	null	Filter by type (`FACT`, `DECISION`, etc.)
`lock_in_state`	string	null	Filter by state (`drifting`, `fluid`, `settled`)
`limit`	int	50	Maximum nodes to return

Response: Array of node objects (same schema as individual node)

curl Example:
```bash
curl "http://localhost:8000/memory/nodes?node_type=FACT&lock_in_state=settled&limit=10"
```

POST /memory/nodes/{node_id}/access

Record an access to a memory node. Increases access count and updates lock-in coefficient.

Path Parameters:

Parameter	Type	Description
`node_id`	string	Node identifier

Response:

Field	Type	Description

`status`	string	`"accessed"`
`node_id`	string	Node ID
`new_access_count`	int	Updated access count
`new_lock_in`	float	Updated lock-in coefficient
`new_lock_in_state`	string	Updated lock-in state

curl Example:
```bash
curl -X POST http://localhost:8000/memory/nodes/node_abc123/access
```

Example Response:
```json
{
  "status": "accessed",
  "node_id": "node_abc123",
  "new_access_count": 5,
  "new_lock_in": 0.35,
  "new_lock_in_state": "fluid"
}
```

POST /memory/nodes/{node_id}/reinforce

Reinforce a memory node. Boosts the lock-in coefficient significantly. Reinforced nodes are protected from pruning.

Path Parameters:

Parameter	Type	Description
`node_id`	string	Node identifier

Response:

Field	Type	Description
`status`	string	`"reinforced"`
`node_id`	string	Node ID
`reinforcement_count`	int	Updated reinforcement count
`new_lock_in`	float	Updated lock-in coefficient
`new_lock_in_state`	string	Updated lock-in state

curl Example:
```bash
curl -X POST http://localhost:8000/memory/nodes/node_abc123/reinforce
```

Example Response:
```json
{
  "status": "reinforced",
```

```
  "node_id": "node_abc123",
  "reinforcement_count": 2,
  "new_lock_in": 0.85,
  "new_lock_in_state": "settled"
}
```

GET /memory/stats

Get memory statistics including lock-in distribution.

Response:

Field	Type	Description
`total_nodes`	int	Total node count
`nodes_by_type`	dict	Nodes grouped by type
`nodes_by_lock_in`	dict	Nodes grouped by lock-in state
`avg_lock_in`	float	Average lock-in coefficient
`total_edges`	int	Total relationship edge count
`drifting_nodes`	int	Count of drifting nodes
`fluid_nodes`	int	Count of fluid nodes
`settled_nodes`	int	Count of settled nodes

curl Example:
```bash
curl http://localhost:8000/memory/stats
```

Example Response:
```json
{
  "total_nodes": 1547,
  "nodes_by_type": {
    "FACT": 823,
    "DECISION": 245,
    "BELIEF": 312,
    "PREFERENCE": 167
  },
  "nodes_by_lock_in": {
    "drifting": 456,
    "fluid": 678,
    "settled": 413
  },
  "avg_lock_in": 0.42,
  "total_edges": 3892,
  "drifting_nodes": 456,
  "fluid_nodes": 678,
  "settled_nodes": 413
}
```

---

5. Extraction Endpoints

These endpoints manage the extraction pipeline (Scribe and Librarian actors).

POST /extraction/trigger

Trigger extraction on content via the Scribe actor.

Request Body:

Field	Type	Required	Default	Description
`content`	string	Yes	-	Content to extract from
`role`	string	No	`"user"`	Role (`user` or `assistant`)
`session_id`	string	No	null	Session identifier
`immediate`	bool	No	true	Process immediately without batching

Response:

Field	Type	Description
`objects_extracted`	int	Number of knowledge objects extracted
`edges_extracted`	int	Number of relationship edges created
`nodes_created`	list[string]	IDs of created memory nodes

curl Example:
```bash
curl -X POST http://localhost:8000/extraction/trigger \
  -H "Content-Type: application/json" \
  -d '{
    "content": "I prefer Python over JavaScript for backend development",
    "role": "user"
  }'
```

Example Response:
```json
{
  "objects_extracted": 2,
  "edges_extracted": 1,
  "nodes_created": ["node_xyz789", "node_abc456"]
}
```

POST /extraction/prune

Trigger synaptic pruning. Removes low-value edges and optionally prunes drifting nodes.

Request Body:

Field	Type	Required	Default	Description
`age_days`	int	No	30	Age threshold in days (min 1)
`confidence_threshold`	float	No	0.3	Confidence threshold (0.0-1.0)
`prune_nodes`	bool	No	true	Also prune drifting nodes
`max_prune_nodes`	int	No	100	Maximum nodes to prune (min 1)

Response:

Field	Type	Description
`edges_pruned`	int	Number of edges removed
`nodes_pruned`	int	Number of nodes removed

curl Example:
```bash
curl -X POST http://localhost:8000/extraction/prune \
  -H "Content-Type: application/json" \
  -d '{
    "age_days": 14,
    "confidence_threshold": 0.2,
    "prune_nodes": true,
    "max_prune_nodes": 50
  }'
```

Example Response:
```json
{
  "edges_pruned": 127,
  "nodes_pruned": 23
}
```

GET /extraction/stats

Get extraction statistics from Scribe and Librarian actors.

Response:

Field	Type	Description
`scribe`	dict	Scribe actor statistics
`librarian`	dict	Librarian actor statistics

curl Example:
```bash
curl http://localhost:8000/extraction/stats
```

Example Response:
```json
```

```
{
  "scribe": {
    "extractions_total": 1547,
    "objects_extracted": 4231,
    "edges_created": 2856,
    "avg_objects_per_extraction": 2.73
  },
  "librarian": {
    "prune_cycles": 12,
    "total_pruned_edges": 892,
    "total_pruned_nodes": 156,
    "last_prune": "2026-01-25T12:00:00Z"
  }
}
```

GET /extraction/history

Get recent extraction history from Scribe.

Query Parameters:

Parameter	Type	Default	Description
`limit`	int	20	Maximum extractions to return

Response:

Field	Type	Description
`extractions`	list	Recent extraction records
`total`	int	Total extraction count

curl Example:
```bash
curl "http://localhost:8000/extraction/history?limit=5"
```

---

6. History Management (Hub API) Endpoints

These endpoints manage conversation history with tiered storage.

POST /hub/session/create

Create a new conversation session.

Request Body:

Field	Type	Required	Default	Description

| `app_context` | string | No | `"terminal"` | Application context identifier |

Response:

Field	Type	Description
`session_id`	string	Session identifier
`started_at`	float	Start timestamp
`ended_at`	float	End timestamp (null if active)
`app_context`	string	Application context

curl Example:
```bash
curl -X POST http://localhost:8000/hub/session/create \
  -H "Content-Type: application/json" \
  -d '{"app_context": "web_app"}'
```

Example Response:
```json
{
  "session_id": "sess_abc123",
  "started_at": 1737817200.0,
  "ended_at": null,
  "app_context": "web_app"
}
```

POST /hub/session/end

End a conversation session.

Query Parameters:

Parameter	Type	Description
`session_id`	string	Session to end

Response:

Field	Type	Description
`success`	bool	Operation success
`session_id`	string	Ended session ID

curl Example:
```bash
curl -X POST "http://localhost:8000/hub/session/end?session_id=sess_abc123"
```

GET /hub/session/active

Get the currently active session.

Response: Session object or `null` if no active session

curl Example:
```bash
curl http://localhost:8000/hub/session/active
```

POST /hub/turn/add

Add a turn to conversation history.

Request Body:

Field	Type	Required	Default	Description
`session_id`	string	No	null	Session ID (uses active if null)
`role`	string	Yes	-	Role (`user` or `assistant`)
`content`	string	Yes	-	Turn content
`tokens`	int	Yes	-	Token count

Response:

Field	Type	Description
`turn_id`	int	Turn identifier
`tier`	string	Tier assigned (`active`, `recent`, `archive`)

curl Example:
```bash
curl -X POST http://localhost:8000/hub/turn/add \
  -H "Content-Type: application/json" \
  -d '{
    "role": "user",
    "content": "What is machine learning?",
    "tokens": 6
  }'
```

GET /hub/active_window

Get the Active Window turns (most recent, highest priority).

Query Parameters:

Parameter	Type	Default	Description

| `session_id` | string | null | Filter by session |
| `limit` | int | 10 | Maximum turns to return |

Response:

Field	Type	Description
`turns`	list	Active window turn objects
`total_tokens`	int	Total token count
`turn_count`	int	Number of turns

curl Example:
```bash
curl "http://localhost:8000/hub/active_window?limit=5"
```

GET /hub/active_token_count

Get token count for Active tier.

Query Parameters:

Parameter	Type	Default	Description
`session_id`	string	null	Filter by session

Response:

Field	Type	Description
`total_tokens`	int	Total token count
`turn_count`	int	Number of turns

curl Example:
```bash
curl http://localhost:8000/hub/active_token_count
```

POST /hub/tier/rotate

Rotate a turn to a new tier.

Request Body:

Field	Type	Description
`turn_id`	int	Turn to rotate
`new_tier`	string	Target tier (`active`, `recent`, `archive`)

Response:

Field	Type	Description
`success`	bool	Operation success

curl Example:
```bash
curl -X POST http://localhost:8000/hub/tier/rotate \
  -H "Content-Type: application/json" \
  -d '{"turn_id": 42, "new_tier": "archive"}'
```

GET /hub/tier/oldest_active

Get the oldest turn in Active tier.

Query Parameters:

Parameter	Type	Default	Description
`session_id`	string	null	Filter by session

Response: Turn object or `null`

curl Example:
```bash
curl http://localhost:8000/hub/tier/oldest_active
```

POST /hub/search

Search conversation history.

Request Body:

Field	Type	Required	Default	Description
`query`	string	Yes	-	Search query
`tier`	string	No	`"recent"`	Tier to search
`session_id`	string	No	null	Filter by session
`limit`	int	No	3	Maximum results
`search_type`	string	No	`"hybrid"`	Search type (`keyword`, `semantic`, `hybrid`)

Response:

Field	Type	Description
`results`	list	Search result objects
`total`	int	Total matching results

curl Example:
```bash
curl -X POST http://localhost:8000/hub/search \
  -H "Content-Type: application/json" \
  -d '{
    "query": "machine learning",
    "tier": "recent",
    "limit": 5
  }'
```

GET /hub/stats

Get history manager statistics.

Response: Dictionary with history manager statistics

curl Example:
```bash
curl http://localhost:8000/hub/stats
```

---

7. Debug Endpoints

These endpoints provide detailed insight into Luna's internal state for debugging.

GET /debug/conversation-cache

Get Luna's conversation cache - the conversation history she is currently aware of.

Response:

Field	Type	Description
`current_turn`	int	Current turn number
`max_turns`	int	TTL for conversation items
`items`	list	Cached conversation items
`total_tokens`	int	Total token count

Conversation Cache Item Schema:

Field	Type	Description
`role`	string	`user`, `assistant`, or `unknown`
`content`	string	Message content
`turn`	int	Turn number
`relevance`	float	Relevance score
`age_turns`	int	Age in turns

curl Example:

```bash
curl http://localhost:8000/debug/conversation-cache
```

Example Response:
```json
{
  "current_turn": 42,
  "max_turns": 10,
  "items": [
    {
      "role": "user",
      "content": "What is machine learning?",
      "turn": 41,
      "relevance": 1.0,
      "age_turns": 1
    },
    {
      "role": "assistant",
      "content": "Machine learning is a subset of AI...",
      "turn": 41,
      "relevance": 1.0,
      "age_turns": 1
    }
  ],
  "total_tokens": 450
}
```

GET /debug/personality

Get Luna's personality system state for debugging.

Response:

Field	Type	Description
`stats`	object	Personality statistics
`patches`	list	Personality patches
`maintenance`	object	Maintenance statistics
`session`	object	Session reflection stats
`mood_state`	string	Current mood
`bootstrap_status`	string	Bootstrap status

Personality Stats Schema:

Field	Type	Description
`total_patches`	int	Total patch count
`active_patches`	int	Active patch count
`average_lock_in`	float	Average lock-in coefficient
`patches_by_topic`	dict	Patches grouped by topic

| `patches_by_lock_in_state` | dict | Patches grouped by lock-in state |

curl Example:
```bash
curl http://localhost:8000/debug/personality
```

GET /debug/context

Get Luna's current context window for debugging.

Response:

Field	Type	Description
`current_turn`	int	Current turn number
`token_budget`	int	Maximum tokens allowed
`total_tokens`	int	Total tokens currently used
`items`	list	Context items
`keywords`	list[string]	Keywords Luna is aware of
`ring_stats`	dict	Statistics per ring

Context Item Schema:

Field	Type	Description
`id`	string	Item ID
`content`	string	Item content
`source`	string	`IDENTITY`, `CONVERSATION`, `MEMORY`, etc.
`ring`	string	`CORE`, `INNER`, `MIDDLE`, `OUTER`
`relevance`	float	Relevance score
`tokens`	int	Token count
`age_turns`	int	Age in turns
`ttl_turns`	int	Time-to-live in turns
`is_expired`	bool	Whether item is expired

curl Example:
```bash
curl http://localhost:8000/debug/context
```

Example Response:
```json
{
  "current_turn": 42,
  "token_budget": 8000,
  "total_tokens": 2450,
  "items": [
    {
      "id": "ctx_001",
      "content": "You are Luna, an AI assistant...",
      "source": "IDENTITY",
```

```
      "ring": "CORE",
      "relevance": 1.0,
      "tokens": 150,
      "age_turns": 0,
      "ttl_turns": -1,
      "is_expired": false
    }
  ],
  "keywords": ["machine learning", "python", "backend"],
  "ring_stats": {
    "CORE": {"items": 3, "tokens": 450},
    "INNER": {"items": 8, "tokens": 1200},
    "MIDDLE": {"items": 5, "tokens": 600},
    "OUTER": {"items": 2, "tokens": 200}
  }
}
```


---

8. Voice Integration Endpoints

These endpoints control voice interaction (speech-to-text and text-to-speech).

POST /voice/start

Start the voice system.

Request Body:

Field	Type	Required	Default	Description
`hands_free`	bool	No	false	Enable hands-free mode (continuous listening)

Response:

Field	Type	Description
`status`	string	`"started"` or `"already_running"`
`message`	string	Status message
`hands_free`	bool	Hands-free mode enabled

curl Example:
```bash
curl -X POST http://localhost:8000/voice/start \
  -H "Content-Type: application/json" \
  -d '{"hands_free": true}'
```

Example Response:
```json
{
  "status": "started",
  "message": "Voice system started in hands-free mode",
```

```
  "hands_free": true
}
```

POST /voice/stop

Stop the voice system.

Response:

Field	Type	Description
`status`	string	`"stopped"` or `"not_running"`
`message`	string	Status message

curl Example:
```bash
curl -X POST http://localhost:8000/voice/stop
```

GET /voice/status

Get voice system status.

Response:

Field	Type	Description
`running`	bool	Voice system is running
`recording`	bool	Currently recording audio
`hands_free`	bool	Hands-free mode enabled
`stt_provider`	string	Speech-to-text provider name
`tts_provider`	string	Text-to-speech provider name
`persona_connected`	bool	Connected to Luna persona
`turn_count`	int	Voice conversation turn count

curl Example:
```bash
curl http://localhost:8000/voice/status
```

Example Response:
```json
{
  "running": true,
  "recording": false,
  "hands_free": false,
  "stt_provider": "whisper",
  "tts_provider": "elevenlabs",
  "persona_connected": true,
```

```
  "turn_count": 12
}
```

POST /voice/listen/start

Start recording user speech (push-to-talk press).

Response:

Field	Type	Description
`status`	string	`"listening"` or `"hands_free"`
`message`	string	Status message

curl Example:
```bash
curl -X POST http://localhost:8000/voice/listen/start
```

POST /voice/listen/stop

Stop recording and process speech (push-to-talk release).

Response:

Field	Type	Description
`status`	string	`"processing"`, `"no_speech"`, or `"hands_free"`
`message`	string	Status message
`transcription`	string	Transcribed text (if speech detected)
`hint`	string	Help hint (when no speech detected)

curl Example:
```bash
curl -X POST http://localhost:8000/voice/listen/stop
```

Example Response:
```json
{
  "status": "processing",
  "message": "Processing your speech",
  "transcription": "What is the weather like today?"
}
```

POST /voice/speak

Speak the given text using TTS.

Request Body:

Field	Type	Description
`text`	string	Text to speak

Response:

Field	Type	Description
`status`	string	`"speaking"`, `"not_running"`, or `"error"`
`text`	string	Text being spoken
`message`	string	Error message (on error)

curl Example:
```bash
curl -X POST http://localhost:8000/voice/speak \
  -H "Content-Type: application/json" \
  -d '{"text": "Hello, how can I help you today?"}'
```

GET /voice/stream

Stream voice status updates via SSE.

SSE Events:

Event	Data Fields	Description
`status`	`connected`, `status`, `running`	Status update
`transcription`	`type`, `text`	User speech transcribed
`response`	`type`, `text`	Luna's response ready
`ping`	`running`	Keepalive (every 10 seconds)

curl Example:
```bash
curl http://localhost:8000/voice/stream \
  -H "Accept: text/event-stream"
```

Example Response Stream:
```
event: status
data: {"connected": true, "status": "idle", "running": true}

event: status
data: {"type": "status", "status": "listening"}

event: transcription
```

```
data: {"type": "transcription", "text": "Hello Luna"}

event: status
data: {"type": "status", "status": "thinking"}

event: response
data: {"type": "response", "text": "Hello! How can I help you?"}

event: status
data: {"type": "status", "status": "speaking"}

event: status
data: {"type": "status", "status": "idle"}

event: ping
data: {"running": true}
```

---

9. Tuning Endpoints

These endpoints allow runtime parameter tuning and evaluation.

GET /tuning/params

List all tunable parameters.

Query Parameters:

Parameter	Type	Default	Description
`category`	string	null	Filter by category

Response:

Field	Type	Description
`params`	list	Parameter list with metadata
`categories`	list	Available categories
`count`	int	Total parameter count

curl Example:
```bash
curl "http://localhost:8000/tuning/params?category=memory"
```

GET /tuning/params/{name}

Get details for a specific parameter.

Path Parameters:

Parameter	Type	Description
`name`	string	Parameter name (use `/` for path-style names)

Response:

Field	Type	Description
`name`	string	Parameter name
`value`	float	Current value
`default`	float	Default value
`bounds`	tuple	(min, max) bounds
`step`	float	Step size for adjustments
`category`	string	Parameter category
`description`	string	Parameter description
`is_overridden`	bool	Whether value differs from default

curl Example:
```bash
curl http://localhost:8000/tuning/params/memory/lock_in_threshold
```

POST /tuning/params/{name}

Set a parameter value.

Path Parameters:

Parameter	Type	Description
`name`	string	Parameter name

Request Body:

Field	Type	Description
`value`	float	New parameter value

Response:

Field	Type	Description
`name`	string	Parameter name
`previous_value`	float	Previous value
`new_value`	float	New value
`eval_score`	float	Evaluation score (if tuning session active)
`iteration`	int	Iteration number (if tuning session active)

curl Example:
```bash
curl -X POST http://localhost:8000/tuning/params/memory/lock_in_threshold \
```

```
  -H "Content-Type: application/json" \
  -d '{"value": 0.7}'
```

POST /tuning/param-reset/{name}

Reset a parameter to its default value.

Path Parameters:

Parameter	Type	Description
`name`	string	Parameter name

Response:

Field	Type	Description
`name`	string	Parameter name
`previous_value`	float	Previous value
`new_value`	float	Default value (now set)
`was_overridden`	bool	Whether it was previously overridden

curl Example:
```bash
curl -X POST http://localhost:8000/tuning/param-reset/memory/lock_in_threshold
```

POST /tuning/session/new

Start a new tuning session for systematic parameter optimization.

Request Body:

Field	Type	Required	Default	Description
`focus`	string	No	`"all"`	Focus area: `memory`, `routing`, `latency`, `context`, `all`
`notes`	string	No	`""`	Session notes

Response:

Field	Type	Description
`session_id`	string	Session identifier
`focus`	string	Focus area
`started_at`	string	Start timestamp
`best_iteration`	int	Best iteration number
`best_score`	float	Best score achieved
`iteration_count`	int	Number of iterations

curl Example:
```bash
curl -X POST http://localhost:8000/tuning/session/new \
  -H "Content-Type: application/json" \
  -d '{"focus": "memory", "notes": "Testing lock-in thresholds"}'
```

GET /tuning/session

Get current tuning session.

Response:

Field	Type	Description
`active`	bool	Whether a session is active
`session_id`	string	Session identifier
`focus`	string	Focus area
`started_at`	string	Start timestamp
`best_iteration`	int	Best iteration number
`best_score`	float	Best score
`iteration_count`	int	Number of iterations
`iterations`	list	Iteration details

curl Example:
```bash
curl http://localhost:8000/tuning/session
```

POST /tuning/session/end

End the current tuning session.

Response:

Field	Type	Description
`ended`	bool	Whether session was ended
`session_id`	string	Session identifier
`best_iteration`	int	Best iteration number
`best_score`	float	Best score achieved
`total_iterations`	int	Total iterations run
`message`	string	Status message (if no session)

curl Example:
```bash
curl -X POST http://localhost:8000/tuning/session/end
```

POST /tuning/eval

Run evaluation to score current parameter settings.

Query Parameters:

Parameter	Type	Default	Description
`category`	string	null	Category to evaluate

Response:

Field	Type	Description
`overall_score`	float	Overall evaluation score
`memory_recall_score`	float	Memory recall performance
`context_retention_score`	float	Context retention performance
`routing_score`	float	Routing decision quality
`avg_latency_ms`	float	Average latency
`p95_latency_ms`	float	95th percentile latency
`total_tests`	int	Total tests run
`passed_tests`	int	Tests passed
`failed_tests`	int	Tests failed

curl Example:
```bash
curl -X POST "http://localhost:8000/tuning/eval?category=memory"
```

GET /tuning/compare

Compare two iterations from the current tuning session.

Query Parameters:

Parameter	Type	Default	Description
`iter1`	int	1	First iteration number
`iter2`	int	latest	Second iteration number

Response:

Field	Type	Description
`iteration_1`	int	First iteration number
`iteration_2`	int	Second iteration number
`score_1`	float	First iteration score
`score_2`	float	Second iteration score
`score_diff`	float	Score difference (iter2 - iter1)
`param_diffs`	dict	Parameter value differences
`metric_diffs`	dict	Metric differences

curl Example:
```bash
curl "http://localhost:8000/tuning/compare?iter1=1&iter2=5"
```

GET /tuning/best

Get parameters from the best iteration.

Response:

Field	Type	Description
`best_iteration`	int	Best iteration number
`best_score`	float	Best score achieved
`params`	dict	Parameter values from best iteration

curl Example:
```bash
curl http://localhost:8000/tuning/best
```

POST /tuning/apply-best

Apply parameters from the best iteration to the current configuration.

Response:

Field	Type	Description
`applied`	int	Number of parameters applied
`best_iteration`	int	Best iteration number
`best_score`	float	Best score

curl Example:
```bash
curl -X POST http://localhost:8000/tuning/apply-best
```

GET /tuning/sessions

List recent tuning sessions.

Query Parameters:

Parameter	Type	Default	Description

| `limit` | int | 10 | Maximum sessions to return |

Response:

Field	Type	Description
`sessions`	list	Session summary objects
`count`	int	Total session count

curl Example:
```bash
curl "http://localhost:8000/tuning/sessions?limit=5"
```

---

10. Ring Buffer Endpoints

These endpoints manage the conversation ring buffer.

GET /api/ring/status

Get current ring buffer status.

Response:

Field	Type	Description
`current_turns`	int	Current turn count in buffer
`max_turns`	int	Maximum turns capacity
`topics`	list[string]	Detected conversation topics (max 10)
`recent_messages`	list[dict]	Recent messages (last 6)

curl Example:
```bash
curl http://localhost:8000/api/ring/status
```

Example Response:
```json
{
  "current_turns": 8,
  "max_turns": 10,
  "topics": ["python", "machine learning", "data processing"],
  "recent_messages": [
    {"role": "user", "content": "How do I use pandas?"},
    {"role": "assistant", "content": "Pandas is a powerful data..."}
  ]
}
```

POST /api/ring/config

Configure the ring buffer size.

Request Body:

Field	Type	Required	Constraints	Description
`max_turns`	int	Yes	2-20	Maximum turns capacity

Response:

Field	Type	Description
`status`	string	`"configured"`
`previous_max_turns`	int	Previous capacity
`new_max_turns`	int	New capacity
`current_turns`	int	Current turn count

curl Example:
```bash
curl -X POST http://localhost:8000/api/ring/config \
  -H "Content-Type: application/json" \
  -d '{"max_turns": 15}'
```

Example Response:
```json
{
  "status": "configured",
  "previous_max_turns": 10,
  "new_max_turns": 15,
  "current_turns": 8
}
```

POST /api/ring/clear

Clear the ring buffer, resetting conversation memory.

Response:

Field	Type	Description
`status`	string	`"cleared"`
`cleared_turns`	int	Number of turns cleared

curl Example:
```bash
curl -X POST http://localhost:8000/api/ring/clear
```

Example Response:

```json
{
  "status": "cleared",
  "cleared_turns": 8
}
```

---

11. System Endpoints

System-level control endpoints.

POST /api/system/relaunch

Trigger a system relaunch. Executes the relaunch script in the background.

Response:

Field	Type	Description
`status`	string	`"restarting"`
`message`	string	Status message

curl Example:
```bash
curl -X POST http://localhost:8000/api/system/relaunch
```

Example Response:
```json
{
  "status": "restarting",
  "message": "Luna Engine is restarting..."
}
```

---

12. SSE Event Reference

This section provides a complete reference for all Server-Sent Events across streaming endpoints.

POST /stream Events

Event	Fields	Description
`token`	`text: string`	Generated token chunk
`done`	`model: string`, `input_tokens: int`, `output_tokens: int`, `latency_ms: float`, `delegated: bool`, `local: bool`, `fallback: bool`	Generation complete with metadata
`error`	`error: string`	Error message

POST /persona/stream Events

Uses typed JSON data format (no named SSE events):

Type	Fields	Description
`context`	`memory: array`, `state: object`	Memory items and engine state (sent first)
`token`	`text: string`	Generated token chunk
`done`	`response: string`, `metadata: object`	Full response and metadata
`error`	`message: string`	Error message

Memory Item Fields:
- `id: string` - Memory node ID
- `content: string` - Content (truncated to 200 chars)
- `type: string` - Node type (FACT, DECISION, etc.)
- `source: string` - Source identifier

State Fields:
- `session_id: string` - Current session
- `is_processing: bool` - Always true during streaming
- `state: string` - Engine state name
- `model: string` - Current model

GET /thoughts Events

Event	Fields	Description
`status`	`connected: bool`, `is_processing: bool`, `goal: string`	Initial connection status
`thought`	`type: "thought"`, `message: string`, `is_processing: bool`, `goal: string`	Internal thought/progress
`phase`	`type: "phase"`, `phase: string`, `is_processing: bool`, `goal: string`	Processing phase change
`step`	`type: "step"`, `step_num: int`, `total_steps: int`, `description: string`, `is_processing: bool`, `goal: string`	Plan step execution
`ping`	`is_processing: bool`, `pending: int`	Keepalive (every 15s)

Thought Message Prefixes:
Prefix	Meaning
`[DIRECT]`	Direct response (no planning)
`[PLAN]`	Creating a plan
`[STEP]`	Executing plan step
`[TOOL]`	Using a tool
`[OK]`	Success
`[ERROR]`	Error occurred
`[ABORT]`	Task aborted

Processing Phases:
Phase	Description
`idle`	No active processing
`planning`	Creating a plan
`observing`	Gathering context
`thinking`	Processing/reasoning
`acting`	Executing actions

GET /voice/stream Events

Event	Fields	Description
`status`	`connected: bool`, `status: string`, `running: bool`	Voice system status
`transcription`	`type: "transcription"`, `text: string`	User speech transcribed
`response`	`type: "response"`, `text: string`	Luna's response ready
`ping`	`running: bool`	Keepalive (every 10s)

Voice Status Values:

Status	Description
`inactive`	Voice system not started
`idle`	Ready, not recording
`listening`	Recording user speech
`thinking`	Processing/generating
`speaking`	Playing TTS audio

---

13. Error Handling

HTTP Status Codes

Code	Meaning	When Returned
`200`	Success	Request completed successfully
`400`	Bad Request	Invalid request body or parameters
`404`	Not Found	Resource not found (node, parameter, etc.)
`500`	Internal Server Error	Operation failed unexpectedly
`503`	Service Unavailable	Engine not ready or required actor unavailable
`504`	Gateway Timeout	Response timeout exceeded

Error Response Format

```json
{
  "detail": "Error message describing what went wrong"
}
```

Common Error Scenarios

Engine Not Ready (503):
```json
{
  "detail": "Engine not ready"
}
```

Actor Not Available (503):
```json
{
```

```
  "detail": "Director not available"
}
```

Resource Not Found (404):
```json
{
  "detail": "Node not found: node_xyz"
}
```

Validation Error (422):
```json
{
  "detail": [
    {
      "loc": ["body", "message"],
      "msg": "field required",
      "type": "value_error.missing"
    }
  ]
}
```

SSE Error Events

Errors during streaming are sent as SSE events:

```
event: error
data: {"error": "Timeout waiting for tokens"}
```

For `/persona/stream`:
```
data: {"type": "error", "message": "Director not available"}
```

---

14. Examples

Basic Conversation

```bash
# Send a message and get response
curl -X POST http://localhost:8000/message \
  -H "Content-Type: application/json" \
  -d '{"message": "What is the capital of France?"}'
```

Streaming Response

```bash
```

```
# Stream response token by token
curl -X POST http://localhost:8000/stream \
  -H "Content-Type: application/json" \
  -H "Accept: text/event-stream" \
  -d '{"message": "Write a haiku about programming"}'
```

Context-Aware Streaming

```bash
# Get context first, then stream response
curl -X POST http://localhost:8000/persona/stream \
  -H "Content-Type: application/json" \
  -H "Accept: text/event-stream" \
  -d '{"message": "What have we discussed?"}'
```

Monitor Thoughts

```bash
# Open a persistent connection to monitor Luna's thinking
curl http://localhost:8000/thoughts \
  -H "Accept: text/event-stream"
```

Create Memory

```bash
# Create a new memory node
curl -X POST http://localhost:8000/memory/nodes \
  -H "Content-Type: application/json" \
  -d '{
    "node_type": "PREFERENCE",
    "content": "User prefers concise explanations",
    "importance": 0.8
  }'
```

Search Memory

```bash
# Get settled facts
curl "http://localhost:8000/memory/nodes?node_type=FACT&lock_in_state=settled&limit=10"
```

Reinforce Important Memory

```bash
# Reinforce a memory to protect from pruning
curl -X POST http://localhost:8000/memory/nodes/node_abc123/reinforce
```

Start Voice Session

```bash
# Start voice with hands-free mode
curl -X POST http://localhost:8000/voice/start \
  -H "Content-Type: application/json" \
  -d '{"hands_free": true}'

# Monitor voice events
curl http://localhost:8000/voice/stream \
  -H "Accept: text/event-stream"
```

Tuning Workflow

```bash
# Start tuning session focused on memory
curl -X POST http://localhost:8000/tuning/session/new \
  -H "Content-Type: application/json" \
  -d '{"focus": "memory"}'

# Adjust a parameter
curl -X POST http://localhost:8000/tuning/params/memory/lock_in_threshold \
  -H "Content-Type: application/json" \
  -d '{"value": 0.65}'

# Run evaluation
curl -X POST http://localhost:8000/tuning/eval

# Compare iterations
curl "http://localhost:8000/tuning/compare?iter1=1&iter2=2"

# Apply best settings
curl -X POST http://localhost:8000/tuning/apply-best

# End session
curl -X POST http://localhost:8000/tuning/session/end
```

Health Check Script

```bash
#!/bin/bash
# Check if Luna is healthy
response=$(curl -s http://localhost:8000/health)
status=$(echo $response | jq -r '.status')

if [ "$status" = "healthy" ]; then
  echo "Luna is running"
  exit 0
else
  echo "Luna is not healthy: $response"
  exit 1
fi
```

JavaScript Client Example

```javascript
// Streaming with fetch API
async function talkToLuna(message) {
  const response = await fetch('http://localhost:8000/persona/stream', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ message })
  });

  const reader = response.body.getReader();
  const decoder = new TextDecoder();

  while (true) {
    const { done, value } = await reader.read();
    if (done) break;

    const text = decoder.decode(value);
    const lines = text.split('\n');

    for (const line of lines) {
      if (line.startsWith('data: ')) {
        const data = JSON.parse(line.slice(6));

        switch (data.type) {
          case 'context':
            console.log('Memory:', data.memory);
            console.log('State:', data.state);
            break;
          case 'token':
            process.stdout.write(data.text);
            break;
          case 'done':
            console.log('\n---');
            console.log('Metadata:', data.metadata);
            break;
          case 'error':
            console.error('Error:', data.message);
            break;
        }
      }
    }
  }
}

// Monitor thoughts with EventSource
const thoughts = new EventSource('http://localhost:8000/thoughts');

thoughts.addEventListener('thought', (e) => {
  const data = JSON.parse(e.data);
  console.log(`[${data.is_processing ? 'THINKING' : 'IDLE'}] ${data.message}`);
});
```

```
thoughts.addEventListener('phase', (e) => {
  const data = JSON.parse(e.data);
  console.log(`Phase: ${data.phase}`);
});
```

Endpoint Summary

Category	Count	Description
Core	7	Health, status, message, abort, interrupt, thoughts
Streaming (SSE)	4	/stream, /persona/stream, /thoughts, /voice/stream
WebSocket	1	/ws/orb
Ring Buffer	3	Status, config, clear
Memory	13	Node CRUD, access, reinforce, search, smart-fetch, add-edge, trace
Extraction	4	Trigger, prune, stats, history
Hub/History	10	Session, turn, tier, search management
Voice	7	Start, stop, status, listen, speak, stream
Tuning	12	Parameters, sessions, eval, compare
Clusters	4	Stats, list, get, constellation
Debug	3	Conversation cache, personality, context
Slash Commands	19	Health, find-person, stats, search, recent, etc.
LLM Provider	3	Providers, current, switch
System	2	Relaunch, set-app-context
Total Engine API	**74**	
MCP API Proxy	25	Engine API proxy for Claude Desktop
MCP Tools	41	Filesystem, memory, session, state, git, forge	

MCP Integration

MCP Server (FastMCP)

Location: `/src/luna_mcp/server.py`
Protocol: stdio (for Claude Desktop)
Name: `Luna-Hub-MCP-V1`
Port (API Proxy): 8742

MCP Tool Categories (41 Total)

Category	Count	Tools
Filesystem	3	luna_read, luna_write, luna_list
Memory	9	luna_smart_fetch, memory_matrix_search, memory_matrix_add_node, memory_matrix_add_edge, memory_matrix_get_context, memory_matrix_trace, luna_save_memory, luna_start_session, luna_record_turn
Session	5	luna_end_session, luna_get_current_session, luna_auto_session_status, luna_flush_session, luna_detect_context
State	2	luna_get_state, luna_set_app_context

| **Git** | 2 | luna_git_sync, luna_git_status |
| **Persona Forge** | 20 | forge_load, forge_assay, forge_gaps, forge_mint, forge_export, forge_status, forge_list_sources, forge_read_raw, forge_add_example, forge_add_batch, forge_search, forge_read_matrix, forge_read_turns, character_list, character_load, character_modulate, character_save, character_show, vk_run, vk_list, vk_probes |

Auto-Session Recording

The MCP layer implements automatic session recording:
- Sessions start automatically on first tool activity
- Turns are buffered and flushed every 4 turns
- Sessions end after 5 minutes of inactivity
- Extraction is triggered on session end

Changelog

Version	Date	Description
3.0.0	Jan 30, 2026	v3.0 Bible Audit - Updated endpoint counts (74 total), added MCP section
2.4.0	Jan 25, 2026	Initial release of Part XV: API Reference